

从复现到改进

让 DatawiseAgent 的任务适配、长交互和资源预算可控

汇报人：於之翔，组员：冉子易张恒

2026.6.10

复现以后，三条值得优化的线

方向	核心问题
1. 任务适配	不同 benchmark 的失败源并不一致。InfiAgentBench 主要暴露答案名、统计口径、预处理顺序和最终格式不稳定；DataModeling 则更多暴露 submission 合法但预测低信息、metric 和 manifest 不完整等问题。因此 harness 不能只做统一 prompt，而要先识别任务类型和错误源。
2. 结构化时间换性能	长交互只有在每一轮都有明确检查目标时才有意义。我们需要把 planning、执行、debug、reformat 拆成可验证环节，失败时只修失败点，并用 final block / verifier 避免从长日志中漏抽或错抽答案。
3. 资源感知调度	强模型并不一定更慢，小模型也不一定更省。资源优化的核心是把确定性 schema、contract、format checker 前置，把强模型留给高风险推理和复杂代码生成，同时减少重复探索和无效对话。

Takeaway: 三条线不是堆模块：先识别任务错法，再把交互结构化，最后按风险决定模型、记忆和预算。

Part I

Task-Specific Adaptation

不同任务错法不同，harness 系统根据具体问题设计

Part I Baseline: 两个 benchmark 的错法不一样

InfiAgentBench baseline / strong reference

Setting	ABQ	PASQ	UASQ
Qwen3-32B full latest	81.74%	86.21%	86.43%
Qwen3.5-35B-A3B baseline	86.38%	90.16%	90.35%

DataModeling baseline / strong reference

Run	Complete	Perf.	Avg time
qwen25	68/74	0.4290	760.25s
qwen35b-a3b-full	73/74	0.5318	288.60s

共同结论

通过强模型，了解答案正确率上限，同时也说明好的理解和规划可以在正确率及耗时上获得高效提升。我们的优化目标就是通过整合系统、记忆等 harness 技巧，让弱模型也能实现强模型的正确率。但针对不同任务，我们需要分别分析他们的错误源。

Part I Analysis: 任务分类分层路由解决口径漂移

核心问题

代码能跑， 但答案口径不稳定

同一张表、同一个问题，模型可能在过滤条件、统计方法、小数位或最终格式上发生轻微漂移；这些错误不一定报错，但会直接影响闭式评分。

Task-Aware harness

1. 先用 level / concepts / keywords / format 识别题型；
2. 不同题型路由到对应 Harness Protocol；
3. protocol 约束执行，并要求可复现证据；
4. verifier 检查输出是否符合任务契约。

answer name

filter

rounding

statistical method

final format

Takeaway: 重点不是让模型“知道答案”，而是把题目口径显式化为 execution contract：任务决定协议，协议约束执行，验证决定能否输出。

Part I Scheme: Task Router → Harness Protocols

路由信号

- ▶ level: easy / medium / hard
- ▶ concepts: 统计、相关性、特征、ML
- ▶ question keywords
- ▶ expected format

收束目标

不是统一大 prompt，
而是让不同题型进入不同协议，再
由 verifier 和 Final Block 收束到稳
定答案通道。

Illustration Placeholder
Task Router → Protocol Cards → Verification Gate

Stats

Corr

Feature

ML

Format

Takeaway: 后续替换为 GPT-image; 核心逻辑: 分类路由 → 协议约束 → 验证收束。

Part I Scheme: DataModeling 从 protocol 到 quality-backed

基于 InfiAgentBench 已有的改动

- ▶ Contract / Profile: 固定 metric、列、行数、target;
- ▶ Task Router: tabular、text、time-series、multi-output;
- ▶ Submission Validator: schema、ID、概率范围、sample copy;
- ▶ Quality Gate: manifest、fallback、常数预测、低信息 final。

依旧存在的问题

- ▶ 结构合法但只是 fallback;
- ▶ 多分类塌缩成单类;
- ▶ 回归/概率输出全局均值;
- ▶ CSV 可评, 但没有 validation 和选择依据。

Takeaway: 这一步不替模型建模, 只把提交前能检查清楚的内容先检查清楚。

Part I Result: Task-Aware harness 带来的直接收益

Setting	Split	ABQ	PASQ	UASQ
Qwen3-32B baseline	medium	83.72%	86.19%	86.00%
Harness rerender	medium	91.95%	93.06%	93.42%
Qwen3-32B baseline	hard	66.13%	78.49%	82.99%
Harness rerender	hard	78.41%	84.75%	87.19%

medium

ABQ 提升 **8.23 个百分点**，说明格式约束和结果抽取能直接减少可避免错误。

hard

ABQ 提升 **12.28 个百分点**，但平均分支数升至 **2.97**，复杂题仍需要控制搜索成本。

答案通道

format error / reformat missing / final block missing = 0，说明结构化答案链路有效。

Part I Result: DataModeling 的结果和边界

Setting	Complete	Perf.	Avg time
qwen25 baseline	68/74	0.4290	760.25s
qwen35b-a3b-full	73/74	0.5318	288.60s
qwen3-vl-8b old harness	74/74	0.3127	799.09s
qwen3-vl-32b old harness	74/74	0.2873	297.49s

+0.1028
qwen35b vs qwen25

74/74
qwen3-vl completion

0.45–0.50
修复典型模式后的目标区间

qwen3-vl 的问题不是不能交文件，而是合法 CSV 里常有均值、常数、fallback 或 manifest 缺失。

Takeaway: DataModeling 已经证明强模型与工程约束有收益；弱模型暴露的是 completion 到 performance 的关键断点。

Part I Insight: DataModeling 不是单一难度任务

有逻辑可循的任务

Titanic、Santander 这类 schema / metric 清晰的 tabular 任务，contract 与 validator 能帮助模型走到有效路径。

低分集中模式

多分类塌缩、均值回归、文本错路由、多输出常数、时间/分组任务错当普通表格。

下一步抓手

manifest-only repair、分级 early-stop、任务族模板、低信息预测拦截。

代表复测	分数	说明
8B Titanic	0.7877	有效 tabular submission; manifest 需要短修复
32B Titanic	0.7709	大模型不自动更好，仍受流程厚度影响
32B Santander	0.8116	预测质量可用，耗时和 manifest 是主要瓶颈

Takeaway: 弱模型低分帮助定位：哪些题需轻约束，哪些题必须进入质量驱动流程。

Failure Insights: 负结果告诉我们怎么设计系统

1. Harness 要适配模型

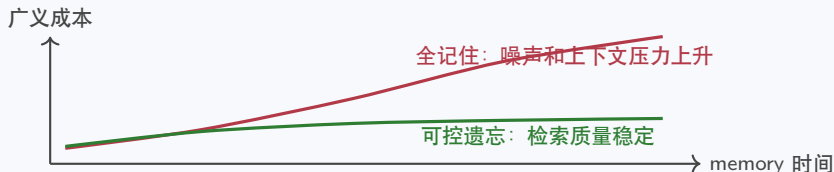
弱模型不能配过厚、过死的流程；目标是按风险调度，而不是 full-everywhere。

2. 记忆不能全记住

旧记忆、低价值记忆会降低信噪比；记忆系统需要重要性、时效性和遗忘机制。

3. Training-free 有上限

DataModeling 跨任务难度大；工程能改路径和验证，但不能完全替代强模型或后训练。



Takeaway: 失败 insight 把边界讲清楚：模型能力、记忆质量和 harness 厚度必须一起调。

Part II

Structured Time-for-Performance

多花时间不是多问几轮，而是把时间变成可验证过程

Part II Baseline: 原作现象与复现后的问题

原作里有用的现象

弱模型可以靠 notebook-style 长交互补一部分能力：

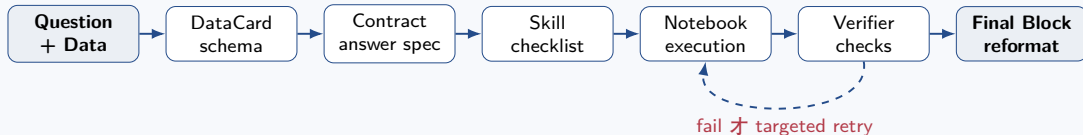
- ▶ planning;
- ▶ code execution;
- ▶ debugging;
- ▶ repair 后继续推进。

复现后看到的问题

- ▶ 长交互不一定更好;
- ▶ full harness 在 hard split 上可能退化;
- ▶ 多分支搜索可能选错;
- ▶ verifier 可能过度阻断;
- ▶ final block 混进 code cell 会卡住 debug loop。

Takeaway: “时间换性能” 不能理解成让小模型盲目多轮规划；多出来的时间必须服务于明确的检查、修复和停止条件。

Part II Scheme: 把长交互拆成可检查环节



执行前： 固定数据结构、答案契约、禁止动作。

执行后： 只在 verifier fail 后短上下文修复；复杂题才启用多候选执行与分支选择，不把完整日志反复交给模型。

Takeaway： 这条线的核心不是模块数量，而是每一轮交互都有输入、检查和停止条件。

Part II Result & Analysis

Setting	ABQ	Hard ABQ	Avg runtime
baseline + original reformat	48.94%	44.00%	30.24s
harness_light	51.57%	47.32%	37.03s
harness_full	52.83%	49.44%	35.79s

light-first

默认只加最关键的结构化约束和 finalizer，让大多数题先稳定完成。

targeted retry

只有 verifier fail 或高风险题，才追加修复、搜索或升级模型。

hard-aware

hard 题不能只加分支数量，还要能做分支选择和语义复算。

Part III

Resource-Aware Scheduling

如何在不改变效率的前提下降低成本消耗

Part III Baseline: 用 EqTok 重设资源指标

等效 token

$$\text{EqTok} = T_{32B} + 0.5T_{14B} + 0.25T_{8B}$$

基于 qwen 官方平台定价 1: 2: 4 换算等价 token。

32B baseline 指标	数值
ABQ	81.74%
PASQ	86.21%
UASQ	86.43%
调用次数	约 1,621
EqTok	5.961M

Takeaway: 这一部分只讨论 32B / 14B / 8B 的资源方案，避免把强模型参考和资源路由候选混在一起。

Part III Scheme A: 小模型使用规则

阶段	默认资源	使用原则
Schema / Contract / Verifier / Finalizer	deterministic 优先	能程序化就不问模型；同一 CSV 的 schema 和列映射复用。
Easy planning / routing	8B / 14B / template	短上下文、低风险、格式化强的子任务交给小模型或模板。
Medium planning/debug	14B	用契约固定口径，verifier fail 后只修失败点。
Medium/hard code generation	32B	代码生成和复杂统计口径仍保持强模型主导。
Hard reasoning / fallback	32B thinking	只有高风险或多次校验失败才启用。

Takeaway: 模型路由不是为了全程用小模型，而是把强模型留给真的需要强推理的阶段。

Part III Scheme B: 减少模型对话发生

机制	作用	影响
deterministic artifacts	schema、contract、finalizer 尽量程序化，不反复询问模型。	降低调用次数
contract-to-operation path	从答案契约直接生成操作计划，减少自由探索。	降低 token 和偏题
one-shot execution	低风险题一次模型调用完成。	降低时间
verifier-gated retry	失败才短上下文修复，不把完整日志再喂回去。	控制 retry 成本
artifact reuse	同一数据文件复用 schema 和列名规范化。	降低重复探索

Takeaway: 这比狭义 solver 更 general: 目标不是覆盖固定统计公式，而是减少不必要的模型对话。

Part III Result: 资源消融真实结果

方案	ABQ	EqTok	调用次数
32B baseline	81.74%	5.961M	1,621
小模型规则	78.42%	3.517M	1,857
小模型规则 + 对话减少	77.87%	2.983M	1,439

强模型不一定更慢

强模型可能减少 debug / repair 轮数，所以总时间可能接近甚至更短。

小模型不一定更划算

小模型只有在简单、明确、短上下文子任务上才适合。

核心是风险调度

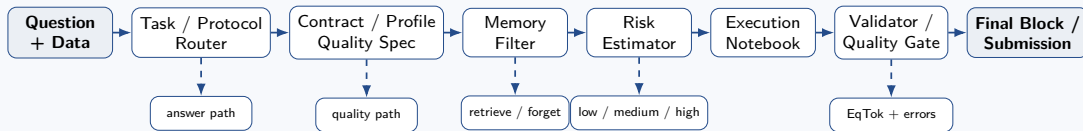
low risk 用 deterministic / small instruct; high risk 才用 32B thinking。

Final Integration

Resource-Aware DatawiseAgent

从三个局部改进到一个统一系统

融合架构：任务、时间、记忆和资源在同一条链上



Takeaway: 任务决定检查项，记忆决定上下文质量，验证决定是否继续，风险决定由谁来做。

融合的预期

1. 失败模式不是统一的

InfiAgentBench 错在答案口径和统计协议；
DataModeling 错在 submission、metric、
manifest 和低信息预测。

2. Completion $\neq$ Performance

qwen3-vl 能做到 74/74 结构完成，但
performance 低，说明合法 CSV 还必须经过
quality gate。

3. 记忆和 harness 都要有选择

全记住会降低检索信噪比；full harness 也会
让弱模型负担过重。关键是保留高价值上下文。

4. 最终系统应该做风险调度

low risk 用 deterministic / small instruct,
medium risk 用 small model + verifier, high
risk 用 strong model + targeted fallback。

感谢聆听